Caching with Redis

Why it matters

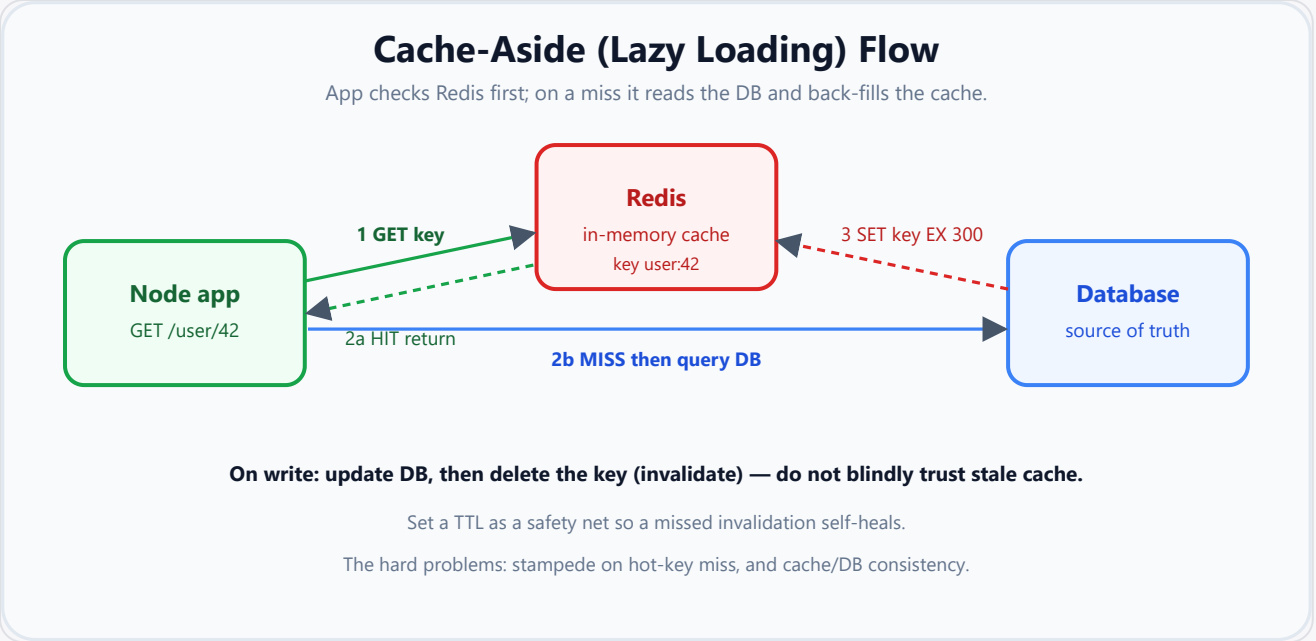
The fastest query is the one you never run. Every time your API can answer from a fast in-memory store instead of hitting the primary database, you save tens of milliseconds, cut database load, and buy headroom to serve more traffic on the same hardware. Under real load, **caching is frequently the single highest-leverage performance change you can make** — a well-placed cache in front of a hot, read-heavy endpoint can turn a p99 of 400 ms into 5 ms and drop database CPU by half.

Redis is the default tool for this in the Node world. It's an in-memory data store that reads and writes in microseconds, speaks a simple protocol, and offers exactly the primitives caching needs: key/value with per-key expiry (TTL), atomic counters, pub/sub, and data structures (hashes, sorted sets, lists). It's also the standard place to keep the *shared* state that cluster workers and multiple containers can't hold in local memory — sessions, rate-limit counters, distributed locks. In interviews, caching is where the questions get sharp: anyone can call "cache it," but the real signal is understanding **invalidation** — famously "one of the two hard things in computer science" — and the failure modes (stampede, penetration, avalanche) that separate a cache that helps from one that takes down your database at 3 a.m.

The concept, simply

A cache is a small, fast store that holds copies of data that's expensive to compute or fetch, keyed so you can look it up quickly. The core question is always **where does the data live and how does it stay correct**. The dominant read pattern is **cache-aside** (a.k.a. lazy loading): your app checks the cache first; on a *hit* it returns immediately; on a

miss it reads the database, stores the result in the cache (with a TTL), and returns it. The cache fills itself on demand — only data that's actually requested gets cached.

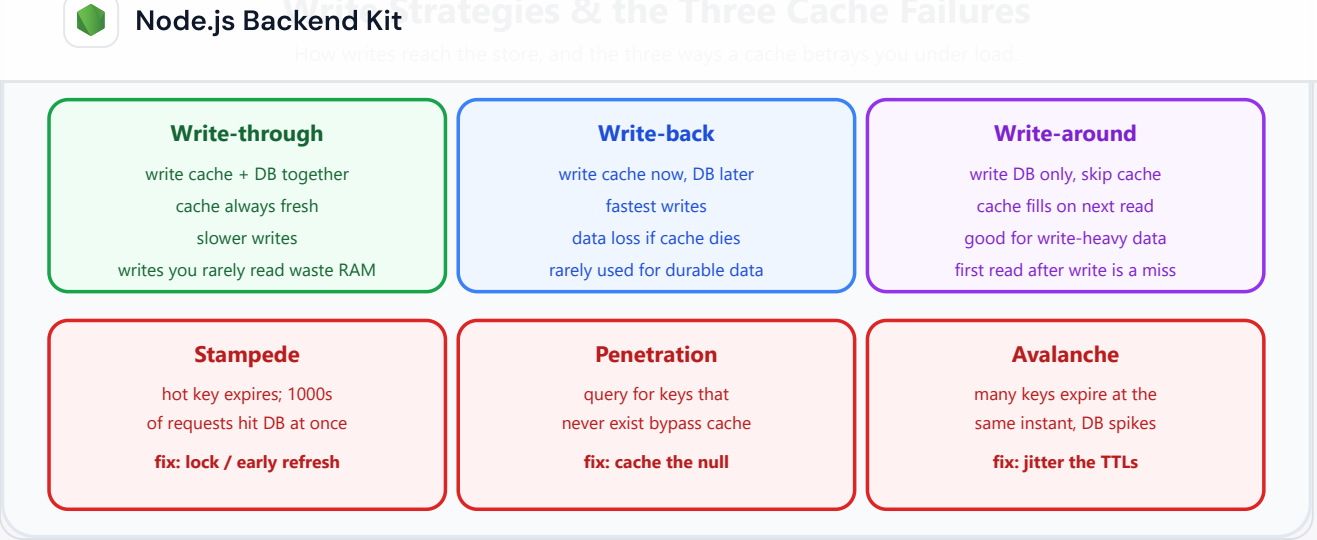


Cache-aside: check Redis, return on hit; on a miss read the DB, write the value back with a TTL, and return it. On writes, update the DB then invalidate the key.

The other half is **writes and invalidation** — keeping the cache from serving stale data after the underlying record changes. The common strategies:

- **Write-through:** write to cache and DB together on every write. Cache is always fresh, but writes are slower and you cache data nobody may read.
- **Write-around:** write only to the DB; let the cache fill on the next read. Good for write-heavy data, but the first read after a write is a miss.
- **Write-back:** write to cache now, flush to DB later. Fastest writes, but you can lose data if the cache dies before flushing — rarely used for durable data.

Most APIs use **cache-aside for reads + explicit invalidation on writes**: after you update the database, you *delete* the cache key (rather than trying to update it), so the next read repopulates fresh. Layered on top, a **TTL** on every key acts as a safety net — even if an invalidation is missed, the stale entry self-heals when it expires.



Write strategies (top) and the three cache failure modes under load (bottom): stampede on a hot-key miss, penetration by keys that never exist, and avalanche when many keys expire together.

How it works

In Node you talk to Redis through a client — `ioredis` (feature-rich, cluster-aware) or the official `redis` (node-redis v4+) package. Both are promise-based and maintain a persistent TCP connection (you create the client once at startup and reuse it — never per request). You mostly use a handful of commands: `GET` / `SET`, `SET key val EX seconds` (set with expiry), `DEL` (invalidate), `INCR` (atomic counter for rate limiting), `EXPIRE`, and for grouped data `HSET` / `HGETALL`. Values are strings/bytes, so objects are stored as `JSON.stringify` and parsed on read.

TTL and eviction. Every cache key should expire. TTL bounds staleness and stops the cache growing without limit. Redis also enforces a memory cap via `maxmemory` + an eviction policy — for a pure cache you typically set `allkeys-lru` (evict least-recently-used keys when full), so Redis behaves like a proper bounded cache rather than filling up and rejecting writes.

Now the three failure modes the diagram names — this is the depth interviewers probe:

- **Cache stampede (thundering herd):** a very popular key expires, and in the microsecond it's gone, thousands of concurrent requests all miss simultaneously and hit the database at once — potentially overwhelming it. Fixes: a short *lock* so only one request recomputes while others wait or serve stale; *early/probabilistic refresh* (recompute just before expiry); or serve-stale-while-revalidate.

- **Cache penetration:** requests for keys that *don't exist* (e.g. random product IDs from a scraper) always miss the cache and always hit the DB, because there's nothing to cache. Fix: cache the negative result (store a short-TTL "null" marker), and/or use a Bloom filter to reject unknown keys.
- **Cache avalanche:** many keys are given the *same* TTL (e.g. warmed at boot with 1-hour expiry) and all expire at the same instant, causing a coordinated flood of misses. Fix: add *jitter* — randomize each TTL by, say, ±10% so expirations spread out.

Sessions and shared state. Beyond caching query results, Redis is the standard session store. In a clustered or multi-container deployment, workers don't share memory, so an in-process session map breaks the moment a user's next request lands on a different worker (see the cluster topic). Storing sessions in Redis — via `connect-redis` for `express-session`, keyed by session ID with a TTL — makes any worker able to serve any user. The same logic makes Redis the home for rate-limit counters (`INCR` + `EXPIRE`) and distributed locks.

Code examples

Cache-aside read with a TTL, using ioredis — the everyday pattern.

JAVASCRIPT

Copy

```
import Redis from 'ioredis';
const redis = new Redis(process.env.REDIS_URL);   // one client, reused

async function getUser(id) {
  const key = `user:${id}`;

  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);          // HIT

  const user = await db.users.findById(id);       // MISS → source of trut
  if (user) {
    // EX = expire seconds; add jitter to avoid avalanche
    const ttl = 300 + Math.floor(Math.random() * 60);
    await redis.set(key, JSON.stringify(user), 'EX', ttl);
  }
```

Write path: update the database, then *invalidate* (delete) the key so the next read repopulates fresh — don't leave stale data behind.

JAVASCRIPT Copy

```
async function updateUser(id, patch) {
  const user = await db.users.update(id, patch);
  await redis.del(`user:${id}`);                    // invalidate; next GET
  return user;
}

// Cache penetration guard: cache the "not found" result briefly so
// repeated lookups of a non-existent id don't keep hammering the DB.
async function getUserSafe(id) {
  const key = `user:${id}`;
  const cached = await redis.get(key);
  if (cached === '__missing__') return null;
  if (cached) return JSON.parse(cached);

  const user = await db.users.findById(id);
  await redis.set(key, user ? JSON.stringify(user) : '__missing__', 'EX',
  return user;
}
```

Stampede protection with a lock — only one request recomputes a hot key; others briefly back off. (`SET NX` = set only if absent, atomic.)

JAVASCRIPT Copy

```
async function getWithLock(key, loader, ttl = 300) {
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);

  // Try to acquire a short lock so only ONE caller rebuilds the value.
  const gotLock = await redis.set(`lock:${key}`, '1', 'EX', 10, 'NX');
  if (!gotLock) {
    await new Promise((r) => setTimeout(r, 50));    // let the winner fill
```

```
    return getWithLock(key, loader, ttl);          // retry the cache
  try {
    const value = await loader();                  // the single DB hit
    await redis.set(key, JSON.stringify(value), 'EX', ttl);
    return value;
  } finally {
    await redis.del(`lock:${key}`);
  }
}
```

Sessions in Redis so any cluster worker / container can authenticate a user.

JAVASCRIPT Copy

```
import session from 'express-session';
import { RedisStore } from 'connect-redis';
import Redis from 'ioredis';

const store = new RedisStore({ client: new Redis(process.env.REDIS_URL) })
app.use(session({
  store,                                       // sessions live in Redis, not
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: { httpOnly: true, secure: true, maxAge: 86_400_000 },
}));
```

Common mistakes & gotchas

- **Caching without a TTL.** Keys live forever, memory fills, and any missed invalidation serves stale data indefinitely. Always set an expiry.
- **Updating the cache instead of invalidating it.** Trying to keep the cached copy in sync with every write is error-prone (concurrent writes, partial updates). The robust default is delete-on-write and let the next read refill.
- **Identical TTLs → avalanche.** Warming many keys with the same expiry makes them all die at once. Add jitter.

- **No stampede protection on hot keys.** A single expiring hot key can unleash a thundering herd on the DB. Use a lock, early refresh, or stale-while-revalidate.
- **Ignoring cache penetration.** Lookups for non-existent keys bypass the cache entirely. Cache negatives (short TTL) or filter with a Bloom filter.
- **Creating a Redis client per request.** Connection setup is expensive; create one client at startup and reuse it. Per-request clients exhaust connections.
- **Treating the cache as a system of record.** Redis (as a cache) can evict or be flushed; never store the only copy of durable data in it. The DB is the source of truth.
- **Not handling Redis being down.** If Redis is unreachable, your app should degrade to hitting the DB, not throw 500s. Wrap cache reads so a cache error falls through to the source.
- **Large `KEYS` scans in production.** `KEYS *` blocks Redis (single-threaded). Use `SCAN`, or better, structure keys so you can delete by known key, not by scanning.

Best practices

- **Default to cache-aside + delete-on-write + TTL.** It's simple, robust, and covers most read-heavy APIs.
- **Always expire keys, and jitter TTLs** to prevent synchronized expiry.
- **Set `maxmemory` + `allkeys-lru`** so Redis behaves as a bounded cache and evicts cold data instead of failing writes.
- **Protect hot keys** with a rebuild lock or early refresh; cache negatives to stop penetration.
- **Use one shared client** (ioredis/node-redis) created at startup; enable reconnection and command timeouts.
- **Fail open.** On a cache error, fall back to the database so a Redis blip doesn't become an outage.
- **Namespace and version keys** (`user:v2:42`) so you can invalidate a whole shape by bumping the version.
- **Store shared state (sessions, rate limits, locks) in Redis** so it survives across cluster workers and containers.

- **Only cache what's hot and read-heavy.** Caching rarely-read data wastes memory; measure hit rate and evict cold entries.

⭐ Interview lens

Q: Explain the cache-aside pattern.

A: The app checks the cache first. On a hit it returns the cached value; on a miss it reads the database, writes the result into the cache with a TTL, and returns it. Writes update the DB and then delete (invalidate) the key so the next read refills. The cache lazily fills with only the data actually requested.

Q: What's a cache stampede and how do you prevent it?

A: A hot key expires and thousands of concurrent requests all miss at once, flooding the database. Prevent it with a short lock so only one request rebuilds the value while others wait or serve stale, with probabilistic early refresh before expiry, or with stale-while-revalidate. (Interviewers love this because "just cache it" candidates miss it.)

Q: On a write, do you update the cache or delete it? Why?

A: Usually delete (invalidate). Updating the cached copy on every write is fragile under concurrent/partial writes and can leave the cache inconsistent with the DB. Deleting the key means the next read repopulates from the source of truth — simpler and safer. A TTL backs it up in case an invalidation is missed.

Q: Why store sessions in Redis instead of in memory?

A: With cluster or multiple containers, workers don't share memory, so an in-process session is invisible when the user's next request lands on a different worker — they appear logged out. Redis is a shared store any worker can read, keyed by session ID with a TTL. (Same reasoning applies to rate-limit counters and distributed locks.)

A: Penetration = requests for keys that never exist always miss and hit the DB (fix: cache the null / Bloom filter). Avalanche = many keys expire at the same instant causing a flood (fix: jitter TTLs). Stampede = one hot key expires and many concurrent requests rebuild it at once (fix: lock / early refresh). Distinct problems, distinct fixes.

Q: What happens to your app if Redis goes down?
A: It should degrade gracefully — cache reads fail open and fall through to the database, maybe slower but still correct. If your app 500s when Redis is unreachable, the cache became a single point of failure. Wrap cache access so errors don't propagate as request failures.

Cheat-card

- **Cache-aside:** check cache → hit returns; miss → read DB, `SET … EX`, return.
- **On write:** update DB then `DEL` key (invalidate); TTL is the safety net.
- Write strategies: **through** (fresh, slow) / **around** (write-heavy) / **back** (fast, risky).
- Three failures: **stampede** (lock/early refresh), **penetration** (cache null/Bloom), **avalanche** (jitter TTLs).
- Redis clients: `ioredis` / `redis` — **one shared client**, never per request.
- `maxmemory` + `allkeys-lru`; DB is source of truth; **fail open** if Redis is down.
- Redis also holds **shared state**: sessions, rate limits (`INCR` + `EXPIRE`), locks (`SET NX`).

QUICK CHECK — no marks, just for you.

1. Describe the cache-aside (lazy loading) read pattern.

- ◯ Write to cache and DB together on every write so the cache is always fresh

- Check the cache first; on a hit return it; on a miss read the DB, store the result with a TTL, and return it
- Precompute and load every possible key into the cache at startup

Check answer

2. On a write, should you update the cached value or delete (invalidate) the key? Why?

- Update the cache in place, because deleting causes an unnecessary miss
- Delete (invalidate) the key, because updating in place is fragile under concurrent/partial writes; the next read repopulates fresh
- Do nothing to the cache; the TTL alone always keeps it correct
- Write to the cache only and flush to the DB later

Check answer

3. What is a cache stampede (thundering herd), and how do you prevent it?

- Requests for keys that never exist always miss; fix by caching the null result
- Many keys share the same TTL and expire together; fix by adding jitter
- The cache grows without limit; fix by setting maxmemory
- A hot key expires and thousands of concurrent requests all miss and hit the DB at once; fix with a rebuild lock, early refresh, or stale-while-revalidate

Check answer

Previous
Child processes

Next
PM2 & Docker

LEARN MORE

ARTICLES & DOCS

- node-redis guide (connect a Node.js app to Redis)
  redis.io
- node-redis: Production usage (errors, reconnection, timeouts)
  redis.io

VIDEOS



Redis Crash Course
Web Dev Simplified

Hand-picked references for this lesson.